

More Ruby on Rails

Rails Routing from the Outside In

This guide covers the user-facing features of Rails routing.

After reading this guide, you will know:

- How to interpret the code in `config/routes.rb`.
- How to construct your own routes, using either the preferred resourceful style or the `match` method.
- How to declare route parameters, which are passed onto controller actions.
- How to automatically create paths and URLs using route helpers.
- Advanced techniques such as creating constraints and mounting Rack endpoints.

1. The Purpose of the Rails Router

The Rails router matches incoming HTTP requests to specific controller actions in your Rails application based on the URL path. (It can also forward to a [Rack \(rails_on_rack.html\)](#) application.) The router also generates path and URL helpers based on the resources configured in the router.

1.1. Routing Incoming URLs to Code

When your Rails application receives an incoming request, it asks the router to match it to a controller action (aka method). For example, take the following incoming request:

```
GET /users/17
```

If the first matching route is:

```
get "/users/:id", to: "users#show"
```

The request is matched to the `UserController` class's `show` action with `{ id: '17' }` in the `params` hash.

The `to:` option expects a `controller#action` format when passed a string. Alternatively, You can pass a symbol and use the `action:` option, instead of `to:`. You can also pass a string without a `#`, in which case the `controller:` option is used instead to `to:`. For example:

```
get "/users/:id", controller: "users", action: :show
```



Rails uses `snake_case` for controller names when specifying routes. For example, if you have a controller named `UserProfilesController`, you would specify a route to the `show` action as `user_profiles#show`.

1.2. Generating Paths and URLs from Code

The Router automatically generates path and URL helper methods for your application. With these methods you can avoid hard-coded path and URL strings.

For example, the `user_path` and `user_url` helper methods are available when defining the following route:

```
get "/users/:id", to: "users#show", as: "user"
```



The `as:` option is used to provide a custom name for a route, which is used when generating URL and path helpers.

Assuming your application contains this code in the controller:

```
@user = User.find(params[:id])
```

and this in the corresponding view:

```
<%= link_to 'User Record', user_path(@user) %>
```

The router will generate the path `/users/17` from `user_path(@user)`. Using the `user_path` helper allows you to avoid having to hard-code a path in your views. This is helpful if you eventually move the route to a different URL, as you won't need to update the corresponding views.

It also generates `user_url`, which has a similar purpose. While `user_path` generates a relative URL like `/users/17`, `user_url` generates an absolute URL such as `https://example.com/users/17` in the above example.

1.3. Configuring the Rails Router

Routes live in `config/routes.rb`. Here is an example of what routes look like in a typical Rails application. The sections that follow will explain the different route helpers used in this file:

```
Rails.application.routes.draw do
  resources :brands, only: [:index, :show] do
    resources :products, only: [:index, :show]
  end

  resource :basket, only: [:show, :update, :destroy]

  resolve("Basket") { route_for(:basket) }
end
```

Since this is a regular Ruby source file, you can use all of Ruby's features (like conditionals and loops) to help you define your routes.



The `Rails.application.routes.draw do ... end` block that wraps your route definitions is required to establish the scope for the router DSL (Domain Specific Language) and must not be deleted.



Be careful with variable names in `routes.rb` as they can clash with the DSL methods of the router.

2. Resource Routing: the Rails Default

Resource routing allows you to quickly declare all of the common routes for a given resource controller. For example, a single call to `resources` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-resources>) declares all of the necessary routes for the `index`, `show`, `new`, `edit`, `create`, `update`, and `destroy` actions, without you having to declare each route separately.

2.1. Resources on the Web

Browsers request pages from Rails by making a request for a URL using a specific HTTP verb, such as `GET`, `POST`, `PATCH`, `PUT`, and `DELETE`. Each HTTP verb is a request to perform an operation on the resource. A resource route maps related requests to actions in a single controller.

When your Rails application receives an incoming request for:

```
DELETE /photos/17
```

it asks the router to map it to a controller action. If the first matching route is:

```
resources :photos
```

Rails would dispatch that request to the `destroy` action on the `PhotosController` with `{ id: '17' }` in `params`.

2.2. CRUD, Verbs, and Actions

In Rails, resourceful routes provide a mapping from incoming requests (a combination of HTTP verb + URL) to controller actions. By convention, each action generally maps to a specific **CRUD** ([active_record_basics.html#crud-reading-and-writing-data](#)) operation on your data. A single entry in the routing file, such as:

```
resources :photos
```

creates seven different routes in your application, all mapping to the `PhotosController` actions:

HTTP Verb	Path	Controller#Action	Used to
GET	/photos	photos#index	display a list of all photos
GET	/photos/new	photos#new	return an HTML form for creating a new photo
POST	/photos	photos#create	create a new photo
GET	/photos/:id	photos#show	display a specific photo
GET	/photos/:id/edit	photos#edit	return an HTML form for editing a photo
PATCH/PUT	/photos/:id	photos#update	update a specific photo
DELETE	/photos/:id	photos#destroy	delete a specific photo

Since the router uses the HTTP verb *and* path to match inbound requests, four URLs can map to seven different controller actions. For example, the same `photos/` path matches to `photos#index` when the verb is `GET` and `photos#create` when the verb is `POST`.



Order matters in the `routes.rb` file. Rails routes are matched in the order they are specified. For example, if you have a `resources :photos` above a `get 'photos/poll'` the `show` action's route for the `resources` line will be matched before the `get` line. If you want the `photos/poll` route to match first, you'll need to move the `get` line **above** the `resources` line.

2.3. Path and URL Helpers

Creating a resourceful route will also expose a number of helpers to controllers and views in your application.

For example, adding `resources :photos` to the route file will generate these `_path` helpers:

Path Helper	Returns URL
<code>photos_path</code>	<code>/photos</code>
<code>new_photo_path</code>	<code>/photos/new</code>
<code>edit_photo_path(:id)</code>	<code>/photos/:id/edit`</code>
<code>photo_path(:id)</code>	<code>/photos/:id</code>

Parameters to the path helpers, such as `:id` above, are passed to the generated URL, such that `edit_photo_path(10)` will return `/photos/10/edit`.

Each of these `_path` helpers also have a corresponding `_url` helper (such as `photos_url`) which returns the same path prefixed with the current host, port, and path prefix.



The prefix used before `"_path"` and `"_url"` is the route name and can be identified by looking at the "prefix" column of the `bin/rails routes` command output. To learn more see [Listing Existing Routes](#) below.

2.4. Defining Multiple Resources at the Same Time

If you need to create routes for more than one resource, you can save a bit of typing by defining them all with a single call to `resources`:

```
resources :photos, :books, :videos
```

The above is a shortcut for:

```
resources :photos
resources :books
resources :videos
```

2.5. Singular Resources

Sometimes, you have a resource that users expect to have only one (i.e. it does not make sense to have an `index` action to list all values of that resource). In that case, you can use `resource` (singular) instead of `resources`.

The below resourceful route creates six routes in your application, all mapping to the `Geocoders` controller:

```
resource :geocoder
resolve("Geocoder") { [:geocoder] }
```



The call to `resolve` is necessary for converting instances of the `Geocoder` to singular routes through **record identification (form_helpers.html#relying-on-record-identification)**.

Here are all of the routes created for a singular resource:

HTTP Verb	Path	Controller#Action	Used to
GET	/geocoder/new	geocoders#new	return an HTML form for creating the geocoder
POST	/geocoder	geocoders#create	create the new geocoder
GET	/geocoder	geocoders#show	display the one and only geocoder resource
GET	/geocoder/edit	geocoders#edit	return an HTML form for editing the geocoder
PATCH/PUT	/geocoder	geocoders#update	update the one and only geocoder resource
DELETE	/geocoder	geocoders#destroy	delete the geocoder resource



Singular resources map to plural controllers. For example, the `geocoder` resource maps to the `GeocodersController`.

A singular resourceful route generates these helpers:

- `new_geocoder_path` returns `/geocoder/new`

- `edit_geocoder_path` returns `/geocoder/edit`
- `geocoder_path` returns `/geocoder`

As with plural resources, the same helpers ending in `_url` will also include the host, port, and path prefix.

2.6. Controller Namespaces and Routing

In large applications, you may wish to organize groups of controllers under a namespace. For example, you may have a number of controllers under an `Admin::` namespace, which are inside the `app/controllers/admin` directory. You can route to such a group by using a [namespace](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-namespace)

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-namespace>) block:

```
namespace :admin do
  resources :articles
end
```

For `Admin::ArticlesController`, Rails will create the following routes:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	<code>/admin/articles</code>	<code>admin/articles#index</code>	<code>admin_articles_path</code>
GET	<code>/admin/articles/new</code>	<code>admin/articles#new</code>	<code>new_admin_article_path</code>
POST	<code>/admin/articles</code>	<code>admin/articles#create</code>	<code>admin_articles_path</code>
GET	<code>/admin/articles/:id</code>	<code>admin/articles#show</code>	<code>admin_article_path(:id)</code>
GET	<code>/admin/articles/:id/edit</code>	<code>admin/articles#edit</code>	<code>edit_admin_article_path(:id)</code>
PATCH/PUT	<code>/admin/articles/:id</code>	<code>admin/articles#update</code>	<code>admin_article_path(:id)</code>
DELETE	<code>/admin/articles/:id</code>	<code>admin/articles#destroy</code>	<code>admin_article_path(:id)</code>

Note that in the above example all of the paths have a `/admin` prefix per the default convention for `namespace`.

2.6.1. Using Module

If you want to route `/articles` (without the prefix `/admin`) to `Admin::ArticlesController`, you can specify the module with a [scope](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-scope)

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-scope>) block:

```
scope module: "admin" do
  resources :articles
end
```

Another way to write the above:

```
resources :articles, module: "admin"
```

2.6.2. Using Scope

Alternatively, you can also route `/admin/articles` to `ArticlesController` (without the `Admin::` module prefix). You can specify the path with a `scope` block:

```
scope "/admin" do
  resources :articles
end
```

Another way to write the above:

```
resources :articles, path: "/admin/articles"
```

For these alternatives (without `/admin` in path and without `Admin::` in module prefix), the named route helpers remain the same as if you did not use `scope`.

In the last case, the following paths map to `ArticlesController`:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	<code>/admin/articles</code>	<code>articles#index</code>	<code>articles_path</code>
GET	<code>/admin/articles/new</code>	<code>articles#new</code>	<code>new_article_path</code>
POST	<code>/admin/articles</code>	<code>articles#create</code>	<code>articles_path</code>
GET	<code>/admin/articles/:id</code>	<code>articles#show</code>	<code>article_path(:id)</code>
GET	<code>/admin/articles/:id/edit</code>	<code>articles#edit</code>	<code>edit_article_path(:id)</code>
PATCH/PUT	<code>/admin/articles/:id</code>	<code>articles#update</code>	<code>article_path(:id)</code>
DELETE	<code>/admin/articles/:id</code>	<code>articles#destroy</code>	<code>article_path(:id)</code>



If you need to use a different controller namespace inside a `namespace` block you can specify an absolute controller path, e.g: `get '/foo', to: '/foo#index'`.

2.7. Nested Resources

It's common to have resources that are logically children of other resources. For example, suppose your application includes these models:

```
class Magazine < ApplicationRecord
  has_many :ads
end

class Ad < ApplicationRecord
  belongs_to :magazine
end
```

Nested route declarations allow you to capture this relationship in your routing:

```
resources :magazines do
  resources :ads
end
```

In addition to the routes for magazines, this declaration will also route ads to an `AdsController`. Here are all of the routes for the nested `ads` resource:

HTTP Verb	Path	Controller#Action	Used to
GET	<code>/magazines/:magazine_id/ads</code>	<code>ads#index</code>	display a list of all ads for a specific magazine
GET	<code>/magazines/:magazine_id/ads/new</code>	<code>ads#new</code>	return an HTML form for creating a new ad belonging to a specific magazine
POST	<code>/magazines/:magazine_id/ads</code>	<code>ads#create</code>	create a new ad belonging to a specific magazine
GET	<code>/magazines/:magazine_id/ads/:id</code>	<code>ads#show</code>	display a specific ad belonging to a specific magazine

HTTP Verb	Path	Controller#Action	Used to
GET	/magazines/:magazine_id/ads/:id/edit	ads#edit	return an HTML form for editing an ad belonging to a specific magazine
PATCH/PUT	/magazines/:magazine_id/ads/:id	ads#update	update a specific ad belonging to a specific magazine
DELETE	/magazines/:magazine_id/ads/:id	ads#destroy	delete a specific ad belonging to a specific magazine

This will also create the usual path and url routing helpers such as `magazine_ads_url` and `edit_magazine_ad_path`. Since the `ads` resource is nested below `magazines`, The ad URLs require a magazine. The helpers can take an instance of `Magazine` as the first parameter (`edit_magazine_ad_path(@magazine, @ad)`).

2.7.1. Limits to Nesting

You can nest resources within other nested resources if you like. For example:

```
resources :publishers do
  resources :magazines do
    resources :photos
  end
end
```

In the above example, the application would recognize paths such as:

```
/publishers/1/magazines/2/photos/3
```

The corresponding route helper would be `publisher_magazine_photo_url`, requiring you to specify objects at all three levels. As you can see, deeply nested resources can become overly complex and cumbersome to maintain.



The general rule of thumb is to only nest resources 1 level deep.

2.7.2. Shallow Nesting

One way to avoid deep nesting (as recommended above) is to generate the collection actions scoped under the parent - so as to get a sense of the hierarchy, but to not nest the member actions. In other words, to only build routes with the minimal amount of information to uniquely identify the resource.



The "member" actions are the ones that apply to an individual resource and require an ID to identify the specific resource they are acting upon, such as `show`, `edit`, etc. The "collection" actions are the ones that act on the entire set of the resource, such as `index`.

For example:

```
resources :articles do
  resources :comments, only: [:index, :new, :create]
end
resources :comments, only: [:show, :edit, :update, :destroy]
```

Above we use the `:only` option which tells Rails to create only the specified routes. This idea strikes a balance between descriptive routes and deep nesting. There is a shorthand syntax to achieve just that, via the `:shallow` option:

```
resources :articles do
  resources :comments, shallow: true
end
```

This will generate the exact same routes as the first example. You can also specify the `:shallow` option in the parent resource, in which case all of the nested resources will be shallow:

```
resources :articles, shallow: true do
  resources :comments
  resources :quotes
end
```

The articles resource above will generate the following routes:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	/articles/:article_id/comments(.:format)	comments#index	article_comments_path

HTTP Verb	Path	Controller#Action	Named Route Helper
POST	/articles/:article_id/comments(.:format)	comments#create	article_comments_path
GET	/articles/:article_id/comments/new(.:format)	comments#new	new_article_comment_path
GET	/comments/:id/edit(.:format)	comments#edit	edit_comment_path
GET	/comments/:id(.:format)	comments#show	comment_path
PATCH/PUT	/comments/:id(.:format)	comments#update	comment_path
DELETE	/comments/:id(.:format)	comments#destroy	comment_path
GET	/articles/:article_id/quotes(.:format)	quotes#index	article_quotes_path
POST	/articles/:article_id/quotes(.:format)	quotes#create	article_quotes_path
GET	/articles/:article_id/quotes/new(.:format)	quotes#new	new_article_quote_path
GET	/quotes/:id/edit(.:format)	quotes#edit	edit_quote_path
GET	/quotes/:id(.:format)	quotes#show	quote_path
PATCH/PUT	/quotes/:id(.:format)	quotes#update	quote_path
DELETE	/quotes/:id(.:format)	quotes#destroy	quote_path
GET	/articles(.:format)	articles#index	articles_path
POST	/articles(.:format)	articles#create	articles_path
GET	/articles/new(.:format)	articles#new	new_article_path
GET	/articles/:id/edit(.:format)	articles#edit	edit_article_path
GET	/articles/:id(.:format)	articles#show	article_path
PATCH/PUT	/articles/:id(.:format)	articles#update	article_path
DELETE	/articles/:id(.:format)	articles#destroy	article_path

The [shallow](#)

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-shallow>) method with a block creates a scope inside of which every nesting is shallow. This generates the same routes as the previous example:

```

shallow do
  resources :articles do
    resources :comments
    resources :quotes
  end
end

```

There are two options that can be used with `scope` to customize shallow routes - `:shallow_path` and `:shallow_prefix`.

The `shallow_path` option prefixes member paths with the given parameter:

```

scope shallow_path: "sekret" do
  resources :articles do
    resources :comments, shallow: true
  end
end

```

The comments resource here will have the following routes generated for it:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	/articles/:article_id/comments(.:format)	comments#index	article_comments_path
POST	/articles/:article_id/comments(.:format)	comments#create	article_comments_path
GET	/articles/:article_id/comments/new(.:format)	comments#new	new_article_comment_path
GET	/sekret/comments/:id/edit(.:format)	comments#edit	edit_comment_path
GET	/sekret/comments/:id(.:format)	comments#show	comment_path
PATCH/PUT	/sekret/comments/:id(.:format)	comments#update	comment_path
DELETE	/sekret/comments/:id(.:format)	comments#destroy	comment_path

The `:shallow_prefix` option adds the specified parameter to the `_path` and `_url` route helpers:

```

scope shallow_prefix: "sekret" do
  resources :articles do
    resources :comments, shallow: true
  end
end

```

The comments resource here will have the following routes generated for it:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	/articles/:article_id/comments(.:format)	comments#index	article_comments_path
POST	/articles/:article_id/comments(.:format)	comments#create	article_comments_path
GET	/articles/:article_id/comments/new(.:format)	comments#new	new_article_comment_path
GET	/comments/:id/edit(.:format)	comments#edit	edit_sekret_comment_path
GET	/comments/:id(.:format)	comments#show	sekret_comment_path
PATCH/PUT	/comments/:id(.:format)	comments#update	sekret_comment_path
DELETE	/comments/:id(.:format)	comments#destroy	sekret_comment_path

2.8. Routing Concerns

Routing concerns allow you to declare common routes that can be reused inside other resources. To define a concern, use a `concern`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Concerns.html#method-i-concern>) block:

```
concern :commentable do
  resources :comments
end

concern :image_attachable do
  resources :images, only: :index
end
```

These concerns can be used in resources to avoid code duplication and share behavior across routes:

```
resources :messages, concerns: :commentable

resources :articles, concerns: [:commentable, :image_attachable]
```

The above is equivalent to:

```
resources :messages do
  resources :comments
end

resources :articles do
  resources :comments
  resources :images, only: :index
end
```

You can also call `concerns`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Concerns.html#method-i-concerns>) in a `scope` or `namespace` block to get the same result as above. For example:

```
namespace :messages do
  concerns :commentable
end

namespace :articles do
  concerns :commentable
  concerns :image_attachable
end
```

2.9. Creating Paths and URLs from Objects

In addition to using the routing helpers, Rails can also create paths and URLs from an array of parameters. For example, suppose you have this set of routes:

```
resources :magazines do
  resources :ads
end
```

When using `magazine_ad_path`, you can pass in instances of `Magazine` and `Ad` instead of the numeric IDs:

```
<%= link_to 'Ad details', magazine_ad_path(@magazine, @ad) %>
```

The generated path will be something like `/magazines/5/ads/42`.

You can also use `url_for`

(https://api.rubyonrails.org/v8.1.3.1/classes/ActionView/RoutingUrlFor.html#method-i-url_for) with an array of objects to get the above path, like this:

```
<%= link_to 'Ad details', url_for([@magazine, @ad]) %>
```

In this case, Rails will see that `@magazine` is a `Magazine` and `@ad` is an `Ad` and will therefore use the `magazine_ad_path` helper. An even shorter way to write that `link_to` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionView/Helpers/UrlHelper.html#method-i-link_to) is to specify just the object instead of the full `url_for` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/UrlFor.html>) call:

```
<%= link_to 'Ad details', [@magazine, @ad] %>
```

If you wanted to link to just a magazine:

```
<%= link_to 'Magazine details', @magazine %>
```

For other actions, you need to insert the action name as the first element of the array, for `edit_magazine_ad_path`:

```
<%= link_to 'Edit Ad', [:edit, @magazine, @ad] %>
```

This allows you to treat instances of your models as URLs, and is a key advantage to using the resourceful style.



In order to automatically derive paths and URLs from objects such as `[@magazine, @ad]`, Rails uses methods from `ActiveModel::Naming` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActiveModel/Naming.html>) and `ActiveModel::Conversion` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActiveModel/Conversion.html>) modules. Specifically, the `@magazine.model_name.route_key` returns `magazines` and `@magazine.to_param` returns a string representation of the model's `id`. So the generated path may be something like `/magazines/1/ads/42` for the objects `[@magazine, @ad]`.

2.10. Adding More RESTful Routes

You are not limited to the **seven routes** that RESTful routing creates by default. You can add additional routes that apply to the collection or individual members of the collection.

The below sections describe adding member routes and collection routes. The term `member` refers to routes acting on a single element, such as `show`, `update`, or `destroy`. The term `collection` refers to routes acting on multiple, or a collection of, elements, such as the `index` route.

2.10.1. Adding Member Routes

You can add a `member`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-member>) block into the resource block like this:

```
resources :photos do
  member do
    get "preview"
  end
end
```

An incoming GET request to `/photos/1/preview` will route to the `preview` action of `PhotosController`. The resource id value will be available in `params[:id]`. It will also create the `preview_photo_url` and `preview_photo_path` helpers.

Within the `member` block, each route definition specifies the HTTP verb (`get` in the above example with `get 'preview'`). In addition to `get` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-get>), you can use `patch` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-patch>), `put` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-put>), `post` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-post>), or `delete` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-delete>).

If you don't have multiple `member` routes, you can also pass `:on` to a route, eliminating the block:

```
resources :photos do
  get "preview", on: :member
end
```

You can also leave out the `:on` option, this will create the same member route except that the resource id value will be available in `params[:photo_id]` instead of `params[:id]`. Route helpers will also be renamed from `preview_photo_url` and `preview_photo_path` to `photo_preview_url` and `photo_preview_path`.

2.10.2. Adding Collection Routes

To add a route to the collection, use a `collection`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-collection>) block:

```
resources :photos do
  collection do
    get "search"
  end
end
```

This will enable Rails to recognize paths such as `/photos/search` with GET, and route to the `search` action of `PhotosController`. It will also create the `search_photos_url` and `search_photos_path` route helpers.

Just as with member routes, you can pass `:on` to a route:

```
resources :photos do
  get "search", on: :collection
end
```



If you're defining additional resource routes with a symbol as the first positional argument, be mindful that it is not equivalent to using a string. Symbols infer controller actions while strings infer paths.

2.10.3. Adding Routes for Additional New Actions

To add an alternate new action using the `:on` shortcut:

```
resources :comments do
  get "preview", on: :new
end
```

This will enable Rails to recognize paths such as `/comments/new/preview` with GET, and route to the `preview` action of `CommentsController`. It will also create the `preview_new_comment_url` and `preview_new_comment_path` route helpers.



If you find yourself adding many extra actions to a resourceful route, it's time to stop and ask yourself whether you're disguising the presence of another resource.

It is possible to customize the default routes and helpers generated by `resources`, see [customizing resourceful routes section](#) for more.

3. Non-Resourceful Routes

In addition to resourceful routing with `resources`, Rails has powerful support for routing arbitrary URLs to actions. You don't get groups of routes automatically generated by resourceful routing. Instead, you set up each route separately within your application.

While you should usually use resourceful routing, there are places where non-resourceful routing is more appropriate. There's no need to try to force every last piece of your application into a resourceful framework if that's not a good fit.

One example use case for non-resourceful routing is mapping existing legacy URLs to new Rails actions.

3.1. Bound Parameters

When you set up a regular route, you supply a series of symbols that Rails maps to parts of an incoming HTTP request. For example, consider this route:

```
get "photos(/:id)", to: "photos#display"
```

If an incoming GET request of `/photos/1` is processed by this route, then the result will be to invoke the `display` action of the `PhotosController`, and to make the final parameter `"1"` available as `params[:id]`. This route will also route the incoming request of `/photos` to `PhotosController#display`, since `:id` is an optional parameter, denoted by parentheses in the above example.

3.2. Dynamic Segments

You can set up as many dynamic segments within a regular route as you like. Any segment will be available to the action as part of `params`. If you set up this route:

```
get "photos/:id/:user_id", to: "photos#show"
```

This route will respond to paths such as `/photos/1/2`. The `params` hash will be `{ controller: 'photos', action: 'show', id: '1', user_id: '2' }`.



By default, dynamic segments don't accept dots - this is because the dot is used as a separator for formatted routes. If you need to use a dot within a dynamic segment, add a constraint that overrides this - for example, `id: /^[^\/]+/` allows anything except a slash.

3.3. Static Segments

You can specify static segments when creating a route by not prepending a colon to a segment:

```
get "photos/:id/with_user/:user_id", to: "photos#show"
```

This route would respond to paths such as `/photos/1/with_user/2`. In this case, `params` would be `{ controller: 'photos', action: 'show', id: '1', user_id: '2' }`.

3.4. The Query String

The `params` will also include any parameters from the query string. For example, with this route:

```
get "photos/:id", to: "photos#show"
```

An incoming GET request for `/photos/1?user_id=2` will be dispatched to the `show` action of the `PhotosController` class as usual and the `params` hash will be `{ controller: 'photos', action: 'show', id: '1', user_id: '2' }`.

3.5. Defining Default Parameters

You can define defaults in a route by supplying a hash for the `:defaults` option. This even applies to parameters that you do not specify as dynamic segments. For example:

```
get "photos/:id", to: "photos#show", defaults: { format: "jpg" }
```

Rails would match `photos/12` to the `show` action of `PhotosController`, and set `params[:format]` to `"jpg"`.

You can also use a `defaults`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-defaults>) block to define the defaults for multiple items:

```
defaults format: :json do
  resources :photos
  resources :articles
end
```



You cannot override defaults via query parameters - this is for security reasons. The only defaults that can be overridden are dynamic segments via substitution in the URL path.

3.6. Naming Routes

You can specify a name that will be used by the `_path` and `_url` helpers for any route using the `:as` option:

```
get "exit", to: "sessions#destroy", as: :logout
```

This will create `logout_path` and `logout_url` as the route helpers in your application. Calling `logout_path` will return `/exit`.

You can also use `as` to override routing helper names defined by `resources` by placing a custom route definition *before* the resource is defined, like this:

```
get ":username", to: "users#show", as: :user
resources :users
```

This will define a `user_path` helper that will match `/:username` (e.g. `/jane`). Inside the `show` action of `UserController`, `params[:username]` will contain the username for the user.

3.7. HTTP Verb Constraints

In general, you should use the `get`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-get>), `post`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-post>), `put`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-put>), `patch`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-patch>), and `delete` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/HttpHelpers.html#method-i-delete>) methods to constrain a route to a particular verb. There is a `match` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Base.html#method-i-match>) method that you could use with the `:via` option to match multiple verbs at once:

```
match "photos", to: "photos#show", via: [:get, :post]
```

The above route matches GET and POST requests to the `show` action of the `PhotosController`.

You can match all verbs to a particular route using `via: :all`:

```
match "photos", to: "photos#show", via: :all
```



Routing both GET and POST requests to a single action has security implications. For example, the GET action won't check for CSRF token (so writing to the database from GET request is not a good idea. For more information see the [security guide \(security.html#csrf-countermeasures\)](#)). In general, avoid routing all verbs to a single action unless you have a good reason.

3.8. Segment Constraints

You can use the `:constraints` option to enforce a format for a dynamic segment:

```
get "photos/:id", to: "photos#show", constraints: { id: /[A-Z]\d{5}/ }
```

The above route definition requires `id` to be 5 alphanumeric characters long. Therefore, this route would match paths such as `/photos/A12345`, but not `/photos/893`. You can more succinctly express the same route this way:

```
get "photos/:id", to: "photos#show", id: /[A-Z]\d{5}/
```

The `:constraints` option takes regular expressions (as well as any object that responds to `matches?` method) with the restriction that regexp anchors can't be used. For example, the following route will not work:

```
get "/:id", to: "articles#show", constraints: { id: /^d/ }
```

However, note that you don't need to use anchors because all routes are anchored at the start and the end.

For example:

```
get "/:id", to: "articles#show", constraints: { id: /\d.+/ }  
get "/:username", to: "users#show"
```

The above routes would allow sharing the root namespace and:

- route paths that always begin with a number, like `/1-hello-world`, to `articles` with `id` value.
- route paths that never begin with a number, like `/david`, to `users` with `username` value.

3.9. Request-Based Constraints

You can also constrain a route based on any method on the **Request object** ([action_controller_overview.html#the-request-object](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-constraints)) that returns a `String`.

You specify a request-based constraint the same way that you specify a segment constraint. For example:

```
get "photos", to: "photos#index", constraints: { subdomain: "admin" }
```

Will match an incoming request with a path to `admin` subdomain.

You can also specify constraints by using a `constraints` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-constraints>) block:

```
constraints subdomain: "admin" do  
  resources :photos  
end
```

Will match something like `https://admin.example.com/photos`.

Request constraints work by calling a method on the **Request object** ([action_controller_overview.html#the-request-object](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Scoping.html#method-i-constraints)) with the same name as the hash key and then comparing the return value with the hash value. For example: `constraints: { subdomain: 'api' }` will match an `api` subdomain as expected. However, using a symbol `constraints: { subdomain: :api }` will not, because `request.subdomain` returns `'api'` as a `String`.



Constraint values should match the corresponding Request object method return type.

There is an exception for the `format` constraint, while it's a method on the `Request` object, it's also an implicit optional parameter on every path. Segment constraints take precedence and the `format` constraint is only applied when enforced through a hash. For example, `get 'foo', constraints: { format: 'json' }` will match `GET /foo` because the format is optional by default.



You can **use a lambda** like in `get 'foo', constraints: lambda { |req| req.format == :json }` to only match the route to explicit JSON requests.

3.10. Advanced Constraints

If you have a more advanced constraint, you can provide an object that responds to `matches?` that Rails should use. Let's say you wanted to route all users on a restricted list to the `RestrictedListController`. You could do:

```
class RestrictedListConstraint
  def initialize
    @ips = RestrictedList.retrieve_ips
  end

  def matches?(request)
    @ips.include?(request.remote_ip)
  end
end

Rails.application.routes.draw do
  get "*path", to: "restricted_list#index",
    constraints: RestrictedListConstraint.new
end
```

You can also specify constraints as a lambda:

```
Rails.application.routes.draw do
  get "*path", to: "restricted_list#index",
    constraints: lambda { |request| RestrictedList.retrieve_ips.include?(
  (request.remote_ip) }
end
```

Both the `matches?` method and the lambda gets the `request` object as an argument.

3.10.1. Constraints in a Block Form

You can specify constraints in a block form. This is useful for when you need to apply the same rule to several routes. For example:

```
class RestrictedListConstraint
  # ...Same as the example above
end

Rails.application.routes.draw do
  constraints(RestrictedListConstraint.new) do
    get "*path", to: "restricted_list#index"
    get "*other-path", to: "other_restricted_list#index"
  end
end
```

You can also use a `lambda`:

```
Rails.application.routes.draw do
  constraints(lambda { |request| RestrictedList.retrieve_ips.include?(request.remote_ip) }) do
    get "*path", to: "restricted_list#index"
    get "*other-path", to: "other_restricted_list#index"
  end
end
```

3.11. Wildcard Segments

A route definition can have a wildcard segment, which is a segment prefixed with a star, such as `*other`:

```
get "photos/*other", to: "photos#unknown"
```

Wildcard segments allow for something called "route globbing", which is a way to specify that a particular parameter (`*other` above) be matched to the remaining part of a route.

So the above route would match `photos/12` or `/photos/long/path/to/12`, setting `params[:other]` to `"12"` or `"long/path/to/12"`.

Wildcard segments can occur anywhere in a route. For example:

```
get "books/*section/:title", to: "books#show"
```

would match `books/some/section/last-words-a-memoir` with `params[:section]` equals `'some/section'`, and `params[:title]` equals `'last-words-a-memoir'`.

Technically, a route can have even more than one wildcard segment. The matcher assigns segments to parameters in the order they occur. For example:

```
get "*a/foo/*b", to: "test#index"
```

would match `zoo/woo/foo/bar/baz` with `params[:a]` equals `'zoo/woo'`, and `params[:b]` equals `'bar/baz'`.

3.12. Format Segments

Given this route definition:

```
get "*pages", to: "pages#show"
```

By requesting `/foo/bar.json`, your `params[:pages]` will be equal to `'foo/bar'` with the request format of JSON in `params[:format]`.

The default behavior with `format` is that if included Rails automatically captures it from the URL and includes it in `params[:format]`, but `format` is not required in a URL.

If you want to match URLs without an explicit format and ignore URLs that include a format extension, you could supply `format: false` like this:

```
get "*pages", to: "pages#show", format: false
```

If you want to make the format segment mandatory, so it cannot be omitted, you can supply `format: true` like this:

```
get "*pages", to: "pages#show", format: true
```

3.13. Redirection

You can redirect any path to any other path by using the `redirect` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Redirection.html#method-i-redirect>) helper in your router:

```
get "/stories", to: redirect("/articles")
```

You can also reuse dynamic segments from the match in the path to redirect to:

```
get "/stories/:name", to: redirect("/articles/%{name}")
```

You can also provide a block to `redirect`, which receives the symbolized path parameters and the request object:

```
get "/stories/:name", to: redirect { |path_params, req| "/articles/#{path_params[:name].pluralize}" }  
get "/stories", to: redirect { |path_params, req| "/articles/#{req.subdomain}" }
```

Please note that default redirection is a 301 "Moved Permanently" redirect. Keep in mind that some web browsers or proxy servers will cache this type of redirect, making the old page inaccessible. You can use the `:status` option to change the response status:

```
get "/stories/:name", to: redirect("/articles/%{name}", status: 302)
```

In all of these cases, if you don't provide the host (`http://www.example.com`), Rails will take those details from the current request.

3.14. Routing to Rack Applications

Instead of specifying `:to` as a String like `'articles#index'`, which corresponds to the `index` method in the `ArticlesController` class, you can specify any **Rack application (rails_on_rack.html)** as the endpoint for a matcher:

```
match "/application.js", to: MyRackApp, via: :all
```

As long as `MyRackApp` responds to `call` and returns a `[status, headers, body]`, the router won't know the difference between the Rack application and a controller action. This is an appropriate use of `via: :all`, as you will want to allow your Rack application to handle all verbs.



An interesting tidbit - `'articles#index'` expands out to `ArticlesController.action(:index)`, which returns a valid Rack application.



Since procs/lambda's are objects that respond to `call`, you can implement very simple routes (e.g. for health checks) inline, something like: `get '/health', to: ->(env) { [204, {}, ['']] }`

If you specify a Rack application as the endpoint for a matcher, remember that the route will be unchanged in the receiving application. With the following route your Rack application should expect the route to be `/admin`:

```
match "/admin", to: AdminApp, via: :all
```

If you would prefer to have your Rack application receive requests at the root path instead, use `mount` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Base.html#method-i-mount>):

```
mount AdminApp, at: "/admin"
```

3.15. Using `root`

You can specify what Rails should route `'/'` to with the `root` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-root>) method:

```
root to: "pages#main"
root "pages#main" # shortcut for the above
```

You typically put the `root` route at the top of the file so that it can be matched first.



The `root` route primarily handles GET requests by default. But it is possible to configure it to handle other verbs (e.g. `root "posts#index", via: :post`)

You can also use `root` inside namespaces and scopes as well:

```
root to: "home#index"

namespace :admin do
  root to: "admin#index"
end
```

The above will match `/admin` to the `index` action for the `AdminController` and match `/` to `index` action of the `HomeController`.

3.16. Unicode Character Routes

You can specify unicode character routes directly. For example:

```
get "こんにちは", to: "welcome#index"
```

3.17. Direct Routes

You can create custom URL helpers by calling `direct` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/CustomUrls.html#method-i-direct>). For example:

```
direct :homepage do
  "https://rubyonrails.org"
end

# >> homepage_url
# => "https://rubyonrails.org"
```

The return value of the block must be a valid argument for the `url_for` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/UrlFor.html>) method. So, you can pass a valid string URL, Hash, Array, an Active Model instance, or an Active Model class.

```
direct :commentable do |model|
  [ model, anchor: model.dom_id ]
end
```

```
direct :main do
  { controller: "pages", action: "index", subdomain: "www" }
end

# >> main_url
# => "http://www.example.com/pages"
```

3.18. Using `resolve`

The `resolve`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/CustomUrls.html#method-i-resolve>) method allows customizing polymorphic mapping of models. For example:

```
resource :basket

resolve("Basket") { [:basket] }
```

```
<%= form_with model: @basket do |form| %>
  <!-- basket form -->
<% end %>
```

This will generate the singular URL `/basket` instead of the usual `/baskets/:id`.

4. Customizing Resourceful Routes

While the default routes and helpers generated by `resources`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-resources>) will usually serve you well, you may need to customize them in some way. Rails allows for several different ways to customize the resourceful routes and helpers. This section will detail the available options.

4.1. Specifying a Controller to Use

The `:controller` option lets you explicitly specify a controller to use for the resource. For example:

```
resources :photos, controller: "images"
```

will recognize incoming paths beginning with `/photos` but route to the `Images` controller:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	<code>/photos</code>	<code>images#index</code>	<code>photos_path</code>
GET	<code>/photos/new</code>	<code>images#new</code>	<code>new_photo_path</code>
POST	<code>/photos</code>	<code>images#create</code>	<code>photos_path</code>
GET	<code>/photos/:id</code>	<code>images#show</code>	<code>photo_path(:id)</code>
GET	<code>/photos/:id/edit</code>	<code>images#edit</code>	<code>edit_photo_path(:id)</code>
PATCH/PUT	<code>/photos/:id</code>	<code>images#update</code>	<code>photo_path(:id)</code>
DELETE	<code>/photos/:id</code>	<code>images#destroy</code>	<code>photo_path(:id)</code>

For namespaced controllers you can use the directory notation. For example:

```
resources :user_permissions, controller: "admin/user_permissions"
```

This will route to the `Admin::UserPermissionsController` instance.



Only the directory notation is supported. Specifying the controller with Ruby constant notation (e.g. `controller: 'Admin::UserPermissions'`) is not supported.

4.2. Specifying Constraints on `id`

You can use the `:constraints` option to specify a required format on the implicit `id`. For example:

```
resources :photos, constraints: { id: /[A-Z][A-Z][0-9]+/ }
```

This declaration constrains the `:id` parameter to match the given regular expression. The router would no longer match `/photos/1` to this route. Instead, `/photos/RR27` would match.

You can specify a single constraint to apply to a number of routes by using the block form:

```
constraints(id: /[A-Z][A-Z][0-9]+)/ do
  resources :photos
  resources :accounts
end
```



You can use the more **advanced constraints** available in non-resourceful routes section in this context as well.



By default the `:id` parameter doesn't accept dots - this is because the dot is used as a separator for formatted routes. If you need to use a dot within an `:id` add a constraint which overrides this - for example `id: /^[^\/]+/` allows anything except a slash.

4.3. Overriding the Named Route Helpers

The `:as` option lets you override the default naming for the route helpers. For example:

```
resources :photos, as: "images"
```

This will match `/photos` and route the requests to `PhotosController` as usual, *but* use the value of the `:as` option to name the helpers `images_path` etc., as shown:

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	/photos	photos#index	images_path
GET	/photos/new	photos#new	new_image_path
POST	/photos	photos#create	images_path
GET	/photos/:id	photos#show	image_path(:id)
GET	/photos/:id/edit	photos#edit	edit_image_path(:id)
PATCH/PUT	/photos/:id	photos#update	image_path(:id)
DELETE	/photos/:id	photos#destroy	image_path(:id)

4.4. Renaming the `new` and `edit` Path Names

The `:path_names` option lets you override the default `new` and `edit` segment in paths. For example:

```
resources :photos, path_names: { new: "make", edit: "change" }
```

This would allow paths such as `/photos/make` and `/photos/1/change` instead of `/photos/new` and `/photos/1/edit`.



The route helpers and controller action names aren't changed by this option. The two paths shown would have `new_photo_path` and `edit_photo_path` helpers and still route to the `new` and `edit` actions.

It is also possible to change this option uniformly for all of your routes by using a `scope` block:

```
scope path_names: { new: "make" } do
  # rest of your routes
end
```

4.5. Prefixing the Named Route Helpers with `:as`

You can use the `:as` option to prefix the named route helpers that Rails generates for a route. Use this option to prevent name collisions between routes using a path scope. For example:

```
scope "admin" do
  resources :photos, as: "admin_photos"
end

resources :photos
```

This changes the route helpers for `/admin/photos` from `photos_path`, `new_photo_path`, etc. to `admin_photos_path`, `new_admin_photo_path`, etc. Without the addition of `as: 'admin_photos'` on the scoped `resources :photos`, the non-scoped `resources :photos` will not have any route helpers.

To prefix a group of route helpers, use `:as` with `scope`:

```
scope "admin", as: "admin" do
  resources :photos, :accounts
end

resources :photos, :accounts
```

Just as before, this changes the `/admin` scoped resource helpers to `admin_photos_path` and `admin_accounts_path`, and allows the non-scoped resources to use `photos_path` and `accounts_path`.



The `namespace` scope will automatically add `:as` as well as `:module` and `:path` prefixes.

4.6. Using `:as` in Nested Resources

The `:as` option can override routing helper names for resources in nested routes as well. For example:

```
resources :magazines do
  resources :ads, as: "periodical_ads"
end
```

This will create routing helpers such as `magazine_periodical_ads_url` and `edit_magazine_periodical_ad_path` instead of the default `magazine_ads_url` and `edit_magazine_ad_path`.

4.7. Parametric Scopes

You can prefix routes with a named parameter:

```
scope ":account_id", as: "account", constraints: { account_id: /\d+/ } do
  resources :articles
end
```

This will provide you with paths such as `/1/articles/9` and will allow you to reference the `account_id` part of the path as `params[:account_id]` in controllers, helpers, and views.

It will also generate path and URL helpers prefixed with `account_`, into which you can pass your objects as expected:

```
account_article_path(@account, @article) # => /1/article/9
url_for([@account, @article])           # => /1/article/9
form_with(model: [@account, @article])  # => <form action="/1/article/9" ...>
```

The `:as` option is also not mandatory, but without it, Rails will raise an error when evaluating `url_for([@account, @article])` or other helpers that rely on `url_for`, such as `form_with` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionView/Helpers/FormHelper.html#method-i-form_with).

4.8. Restricting the Routes Created

By default, using `resources` creates routes for the seven default actions (`index`, `show`, `new`, `create`, `edit`, `update`, and `destroy`). You can use the `:only` and `:except` options to limit which routes are created.

The `:only` option tells Rails to create only the specified routes:

```
resources :photos, only: [:index, :show]
```

Now, a GET request to `/photos` or `/photos/:id` would succeed, but a POST request to `/photos` will fail to match.

The `:except` option specifies a route or list of routes that Rails should *not* create:

```
resources :photos, except: :destroy
```

In this case, Rails will create all of the normal routes except the route for `destroy` (a DELETE request to `/photos/:id`).



If your application has many RESTful routes, using `:only` and `:except` to generate only the routes that you actually need can cut down on memory use and speed up the routing process by eliminating **unused routes**.

4.9. Translated Paths

Using `scope`, we can alter path names generated by `resources`:

```
scope(path_names: { new: "neu", edit: "bearbeiten" }) do
  resources :categories, path: "kategorien"
end
```

Rails now creates routes to the `CategoriesController`.

HTTP Verb	Path	Controller#Action	Named Route Helper
GET	/kategorien	categories#index	categories_path
GET	/kategorien/neu	categories#new	new_category_path
POST	/kategorien	categories#create	categories_path
GET	/kategorien/:id	categories#show	category_path(:id)
GET	/kategorien/:id/bearbeiten	categories#edit	edit_category_path(:id)
PATCH/PUT	/kategorien/:id	categories#update	category_path(:id)
DELETE	/kategorien/:id	categories#destroy	category_path(:id)

4.10. Specifying the Singular Form of a Resource

If you need to override the singular form of a resource, you can add a rule to ActiveSupport Inflector via [inflections](https://api.rubyonrails.org/v8.1.3.1/classes/ActiveSupport/Inflector.html#method-i-inflections) (<https://api.rubyonrails.org/v8.1.3.1/classes/ActiveSupport/Inflector.html#method-i-inflections>):

```
ActiveSupport::Inflector.inflections do |inflect|
  inflect.irregular "tooth", "teeth"
end
```

4.11. Renaming Default Route Parameter id

It is possible to rename the default parameter name `id` with the `:param` option. For example:

```
resources :videos, param: :identifier
```

Will now use `params[:identifier]` instead of `params[:id]`.

```

      videos GET    /videos(.:format)          videos#index
      POST    /videos(.:format)          videos#create
  new_video GET    /videos/new(.:format)      videos#new
edit_video GET    /videos/:identifier/edit(.:format) videos#edit

```

```
Video.find_by(id: params[:identifier])
```

Instead of

```
Video.find_by(id: params[:id])
```

You can override `ActiveRecord::Base#to_param`

(https://api.rubyonrails.org/v8.1.3.1/classes/ActiveRecord/Integration.html#method-i-to_param) of the associated model to construct a URL:

```

class Video < ApplicationRecord
  def to_param
    identifier
  end
end

```

```

irb> video = Video.find_by(identifier: "Roman-Holiday")
irb> edit_video_path(video)
=> "/videos/Roman-Holiday/edit"

```

5. Inspecting Routes

Rails offers a few different ways of inspecting and testing your routes.

5.1. Listing Existing Routes

To get a complete list of routes available in an application, visit `http://localhost:3000/rails/info/routes` in the **development** environment. You can also execute the `bin/rails routes` command in your terminal to get the same output.

Both methods will list all of your routes, in the same order that they appear in `config/routes.rb`. For each route, you'll see:

- The route name (if any)

- The HTTP verb used (if the route doesn't respond to all verbs)
- The URL pattern to match
- The routing parameters for the route

For example, here's a small section of the `bin/rails routes` output for a RESTful route:

```

      users GET    /users(.:format)      users#index
           POST   /users(.:format)      users#create
  new_user GET    /users/new(.:format)  users#new
edit_user GET    /users/:id/edit(.:format) users#edit

```

The route name (`new_user` above, for example) can be considered the base for deriving route helpers. To get the name of a route helper, add the suffix `_path` or `_url` to the route name (`new_user_path` , for example).

You can also use the `--expanded` option to turn on the expanded table formatting mode.

```

$ bin/rails routes --expanded

--[ Route 1 ]-----
Prefix          | users
Verb            | GET
URI             | /users(.:format)
Controller#Action | users#index
--[ Route 2 ]-----
Prefix          |
Verb            | POST
URI             | /users(.:format)
Controller#Action | users#create
--[ Route 3 ]-----
Prefix          | new_user
Verb            | GET
URI             | /users/new(.:format)
Controller#Action | users#new
--[ Route 4 ]-----
Prefix          | edit_user
Verb            | GET
URI             | /users/:id/edit(.:format)
Controller#Action | users#edit

```

5.2. Searching Routes

You can search through your routes with the `grep` option: `-g` . This outputs any routes that partially match the URL helper method name, the HTTP verb, or the URL path.

```
$ bin/rails routes -g new_comment
$ bin/rails routes -g POST
$ bin/rails routes -g admin
```

If you only want to see the routes that map to a specific controller, there's the controller option: `-c`.

```
$ bin/rails routes -c users
$ bin/rails routes -c admin/users
$ bin/rails routes -c Comments
$ bin/rails routes -c Articles::CommentsController
```



The output from `bin/rails routes` is easier to read if you widen your terminal window until the output lines don't wrap or use the `--expanded` option.

5.3. Listing Unused Routes

You can scan your application for unused routes with the `--unused` option. An "unused" route in Rails is a route that is defined in the `config/routes.rb` file but is not referenced by any controller action or view in your application. For example:

```
$ bin/rails routes --unused
Found 8 unused routes:
```

Prefix	Verb	URI Pattern	Controller#Action
people	GET	/people(/.:format)	people#index
	POST	/people(/.:format)	people#create
new_person	GET	/people/new(/.:format)	people#new
edit_person	GET	/people/:id/edit(/.:format)	people#edit
person	GET	/people/:id(/.:format)	people#show
	PATCH	/people/:id(/.:format)	people#update
	PUT	/people/:id(/.:format)	people#update
	DELETE	/people/:id(/.:format)	people#destroy

5.4. Routes in Rails Console

You can access route helpers using `Rails.application.routes.url_helpers` within the **Rails Console** ([command_line.html#bin-rails-console](#)). They are also available via the **app** ([command_line.html#the-app-and-helper-objects](#)) object. For example:

```
irb> Rails.application.routes.url_helpers.users_path
=> "/users"

irb> user = User.first
=> #<User:0x00007fc1eab81628
irb> app.edit_user_path(user)
=> "/users/1/edit"
```

6. Testing Routes

Rails offers three built-in assertions designed to make testing routes simpler:

- `assert_generates`
(https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_generates)
- `assert_recognizes`
(https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_recognizes)
- `assert_routing`
(https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_routing)

6.1. The `assert_generates` Assertion

`assert_generates` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_generates) asserts that a particular set of options generate a particular path and can be used with default routes or custom routes. For example:

```
assert_generates "/photos/1", { controller: "photos", action: "show", id: "1" }
assert_generates "/about", controller: "pages", action: "about"
```

6.2. The `assert_recognizes` Assertion

`assert_recognizes` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_recognizes) is the inverse of `assert_generates`. It asserts that a given path is recognized and routes it to a particular spot in your application. For example:


```
assert_recognizes({ controller: "photos", action: "show", id: "1" }, "/photos/1")
```

You can supply a `:method` argument to specify the HTTP verb:

```
assert_recognizes({ controller: "photos", action: "create" }, { path: "photos", method: :post })
```

6.3. The `assert_routing` Assertion

The `assert_routing`

(https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Assertions/RoutingAssertions.html#method-i-assert_routing) assertion checks the route both ways. It combines the functionality of both `assert_generates` and `assert_recognizes`. It tests that the path generates the options, and that the options generate the path:

```
assert_routing({ path: "photos", method: :post }, { controller: "photos", action: "create" })
```

7. Breaking Up a Large Route File With `draw`

In a large application with thousands of routes, a single `config/routes.rb` file can become cumbersome and hard to read. Rails offers a way to break up a single `routes.rb` file into multiple small ones using the `draw` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Routing/Mapper/Resources.html#method-i-draw>) macro.

For example, you could add an `admin.rb` file that contains all the routes related to the admin area, another `api.rb` file for API related resources, etc.

```
# config/routes.rb

Rails.application.routes.draw do
  get "foo", to: "foo#bar"

  draw(:admin) # Will load another route file located in `config/routes/admin.rb`
end
```

```
# config/routes/admin.rb

namespace :admin do
  resources :comments
end
```

Calling `draw(:admin)` inside the `Rails.application.routes.draw` block itself will try to load a route file that has the same name as the argument given (`admin.rb` in this example). The file needs to be located inside the `config/routes` directory or any sub-directory (i.e. `config/routes/admin.rb` or `config/routes/external/admin.rb`).



You can use the normal routing DSL inside a secondary routing file such as `admin.rb`, but *do not* surround it with the `Rails.application.routes.draw` block. That should be used in the main `config/routes.rb` file only.



Don't use this feature unless you really need it. Having multiple routing files make it harder to discover routes in one place. For most applications - even those with a few hundred routes - it's easier for developers to have a single routing file. The Rails routing DSL already offers a way to break routes in an organized manner with `namespace` and `scope`.



Feedback

You're encouraged to help improve the quality of this guide.

Please contribute if you see any typos or factual errors. To get started, you can read our **[documentation contributions \(https://edgeguides.rubyonrails.org/contributing_to_ruby_on_rails.html#contributing-to-the-rails-documentation\)](https://edgeguides.rubyonrails.org/contributing_to_ruby_on_rails.html#contributing-to-the-rails-documentation)** section.

You may also find incomplete content or stuff that is not up to date. Please do add any missing documentation for main. Make sure to check **[Edge Guides \(https://edgeguides.rubyonrails.org\)](https://edgeguides.rubyonrails.org)** first to verify if the issues are already fixed or not on the main branch. Check the **[Ruby on Rails Guides Guidelines \(ruby_on_rails_guides_guidelines.html\)](https://edgeguides.rubyonrails.org/ruby_on_rails_guides_guidelines.html)** for style and conventions.

If for whatever reason you spot something to fix but cannot patch it yourself, please **[open an issue \(https://github.com/rails/rails/issues\)](https://github.com/rails/rails/issues)**.

And last but not least, any kind of discussion regarding Ruby on Rails documentation is very welcome on the **[official Ruby on Rails Forum \(https://discuss.rubyonrails.org/c/rubyonrails-docs\)](https://discuss.rubyonrails.org/c/rubyonrails-docs)**.

This work is licensed under a **Creative Commons Attribution-ShareAlike 4.0 International (<https://creativecommons.org/licenses/by-sa/4.0/>)** License

"Rails", "Ruby on Rails", and the Rails logo are trademarks of David Heinemeier Hansson. All rights reserved.